OpenAuthority

Whitepaper

Least-Privilege Trust Infrastructure with
Continuous DNS Verification and Transparency Logging

Version 1.0

Francis G. Booth

CaramelFox Networks LLC

May 25, 2026

Contents

Abstract

OpenAuthority is an experimental infrastructure project exploring least-privilege principles applied to certificate authority trust. Rather than granting unrestricted global issuance authority to trusted roots, OpenAuthority constrains each CA to explicitly verified namespaces, continuously re-validates eligibility through DNS-based ownership proofs, and maintains cryptographically protected transparency logs.

1 Problem Statement

The modern Web Public Key Infrastructure (WebPKI) relies on a hierarchical trust model where certificate authorities possess broad issuance authority. While Certificate Transparency (CT) and projects like Let's Encrypt have significantly improved certificate accessibility and ecosystem visibility, fundamental structural challenges remain.

1.1 Structural Problems

Problem	Description
Excessive Trust Scope	Globally trusted roots can issue certificates for any domain, regardless of legitimate relationship
Static Trust Assertions	Trust persists until explicit revocation—no continuous eligibility verification
Limited Blast Radius Control	CA compromise impacts the entire Internet namespace
Implicit Trust Dependencies	Relying parties trust hundreds of CAs without visibility into authority boundaries

Table 1: Structural problems in current PKI trust models

1.2 Why These Problems Persist

Name Constraints are underutilized. X.509 Name Constraints exist but are rarely deployed. The WebPKI ecosystem evolved around broad authority models; tooling and policies assume unconstrained roots.

CA trust is static by design. Traditional trust stores treat CA inclusion as binary and persistent. There is no mechanism for trust to decay automatically when eligibility conditions change.

Organizational trust boundaries drift. Domain ownership changes through acquisitions, divestitures, and migrations. A CA legitimately authorized in 2020 may have no relationship with that domain in 2025—but its authority persists.

1.3 Why Certificate Transparency Is Not Sufficient

Certificate Transparency provides visibility into issued certificates by requiring CAs to submit certificates to publicly auditable logs. However, CT does not:

- **Constrain issuance authority** — A CT-logged certificate from a compromised CA is still valid
- **Continuously revalidate CA eligibility** — CT logs record issuance events, not ongoing authority boundaries
- **Limit blast radius structurally** — CT detects misissuance; it does not prevent it

OpenAuthority addresses a different layer: governing trust eligibility rather than detecting certificate misissuance after issuance.

1.4 Motivating Scenario

An enterprise operates internal PKI for `corp.example.com`. Over time:

1. A business unit is divested, taking `subsidiary.corp.example.com`
2. The original CA retains issuance capability for the divested domain
3. Years later, the CA's private key is compromised
4. The attacker can issue certificates for domains the enterprise no longer owns

Stale delegation, orphaned trust relationships, and organizational boundary drift are operational realities in enterprise PKI.

2 Core Architectural Principle

Certificate authority trust should be scoped, time-bounded, and continuously verified—not global, permanent, and implicit.

The central innovation is not DNS verification or transparency logging individually—it is that **trust eligibility becomes a continuously evaluated state rather than a permanent binary**.

OpenAuthority does not eliminate trust dependencies; it attempts to reduce authority scope and improve trust freshness.

Concept	**Application**
Zero Trust	No CA is trusted beyond verified eligibility boundaries
Capability Systems	Authority is a constrained capability, not ambient permission
Sandboxing	Name Constraints create cryptographic issuance boundaries
Constrained Delegation	Trust is delegated for specific namespaces only
Automatic Expiry	Authority decays when ownership proofs disappear

Table 2: Security concepts applied to CA trust

3 Threat Model

3.1 Adversary Capabilities

Threat Actor	Capabilities	Mitigation
Compromised CA	Issue certificates within namespace	Name constraints limit scope
Rogue CA Operator	Intentional authority misuse	Continuous verification detects control loss
DNS Hijacker	Temporary DNS control	Multi-resolver verification, probationary periods
Log Tampering	Modify audit history	Hash chaining, Merkle proofs, external anchoring

Table 3: Threat actors and mitigations

3.2 Threat: Stale Delegation

One of the most operationally significant threats:

- CA retains authority after ownership changes
- Former operators retain issuance capability indefinitely
- Orphaned trust relationships persist without review

Continuous DNS verification addresses this directly. When ownership changes, the new owner will not maintain the previous CA's TXT record, causing verification to fail and eligibility to be revoked automatically.

3.3 Security Assumptions

- DNS control is an operational proxy for namespace control, not proof of legal ownership
- Cryptographic hash functions remain collision-resistant
- External timestamp authorities provide independent temporal attestation
- The deployment platform provides adequate isolation and availability

3.4 Out-of-Scope Threats

- Nation-state adversaries with persistent DNS infrastructure control
- Compromise of the OpenAuthority infrastructure itself
- Side-channel attacks on the deployment platform
- Social engineering attacks against domain registrars

4 Design Goals

Goal	Description
Constrained Authority Scope	Mandatory Name Constraints limit each CA to specific domains
Continuous Verification	Re-verify ownership rather than relying on static issuance
Automatic Trust Decay	Eligibility expires when ownership proofs disappear
Observable Trust Decisions	All decisions visible and auditable
Reproducible Exports	Deterministic, independently verifiable trust store exports

Table 4: OpenAuthority design goals

5 Non-Goals

- **WebPKI replacement** — Explores complementary constraints, not wholesale replacement
- **Browser integration** — No browser vendor integration planned
- **Production infrastructure** — Experimental and research-oriented
- **Certificate issuance** — Verifies CA eligibility; does not issue certificates

6 Terminology

Term	Definition
Trust Anchor	A CA certificate included in the trust store as a root of trust
Constrained Root	A trust anchor with mandatory X.509 Name Constraints
Eligibility	Verified right to be included in the trust store for specific namespaces
Namespace	Domain names a constrained root may issue certificates for
Verification	Confirming continued DNS-based ownership proof
Trust Store	Exported set of currently eligible trust anchors
Trust Freshness	The recency and continuity of successful ownership verification for a trust anchor

Table 5: OpenAuthority terminology

All trust anchors in OpenAuthority are constrained roots. Unconstrained CAs are not eligible.

7 Architecture

7.1 System Components

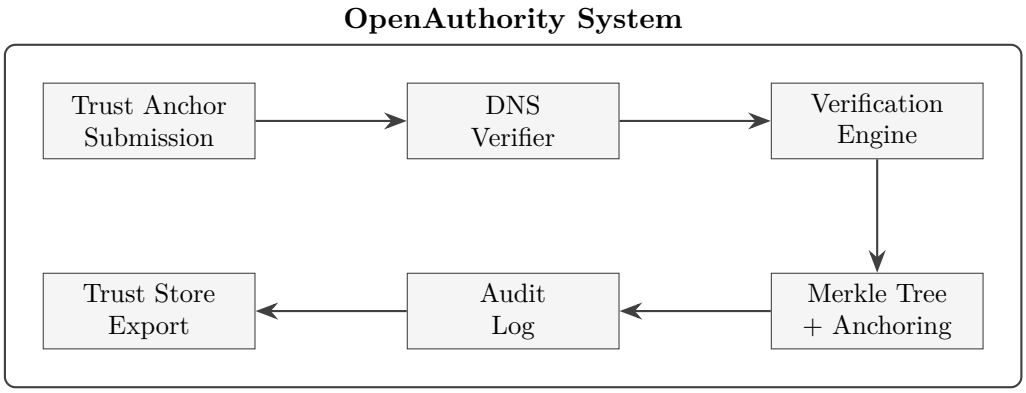

Figure 1: OpenAuthority system components

7.2 Relying Party Model

OpenAuthority assumes relying parties periodically consume signed trust store exports and perform standard X.509 path validation locally. The system does not intercept or modify certificate validation—it governs which trust anchors are included in exported trust stores.

7.3 Trust Anchor Lifecycle

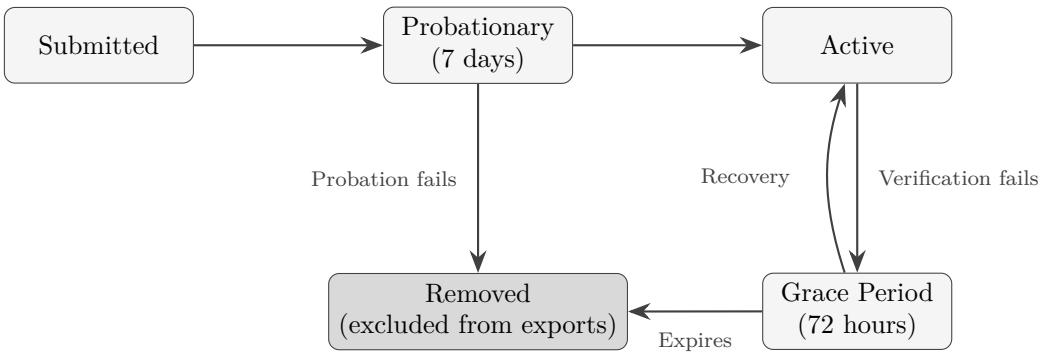

Figure 2: Trust anchor lifecycle state machine

7.4 Lifecycle Example

Normal Operation:

1. Organization generates constrained root with Name Constraints for `corp.example.com`
2. Publishes TXT record at `_openauthority.corp.example.com` with CA fingerprint
3. Verification succeeds every 6 hours for 7 days
4. Trust anchor marked active; included in exports
5. Re-verified every 24 hours

Ownership Change:

6. Organization sells domain; new owner removes TXT record
7. Verification fails

8. 72-hour grace period begins
9. No recovery; trust anchor removed from exports
10. Historical records preserved in audit log

8 Verification Model

8.1 DNS-Based Ownership Verification

Trust anchors prove namespace control via DNS TXT records:

```
_openauthority.example.com. IN TXT "openauthority=sha512:<fingerprint>"
```

DNS control is an operational proxy for namespace control—not proof of legal ownership, corporate authority, or contractual rights.

OpenAuthority currently treats DNS as an operational signaling layer independent of DNSSEC deployment status. DNSSEC integration is considered future work due to uneven global deployment and operational complexity.

8.2 Multi-Resolver Verification

Queries performed across multiple independent resolvers to mitigate single-resolver compromise, caching anomalies, and localized hijacking.

8.3 Verification Schedule and Trust Freshness

Status	Interval	Maximum Freshness Window
Probationary	Every 6 hours	6 hours
Active	Every 24 hours	24 hours

Table 6: Verification schedule

Trust freshness decreases as elapsed time since successful verification increases. The verification intervals define the maximum trust freshness window for each state.

8.4 Failure Semantics

Condition	Treatment
NXDOMAIN	Verification fails
No matching TXT	Verification fails
SERVFAIL / Timeout	Retry with backoff before marking failed
Resolver disagreement	Quorum-based; majority must agree

Table 7: Verification failure semantics

Transient failures do not immediately revoke trust. Persistent failure triggers state transition.

8.5 Name Constraint Enforcement

The effectiveness of constrained roots depends on relying-party enforcement of X.509 Name Constraints. Enforcement behavior may vary across software ecosystems and legacy clients.

9 Trust Store Semantics

9.1 What "Removal" Means

Effect	Description
Future exports exclude the trust anchor	New exports will not contain the removed CA
Existing certificates are not invalidated	Already-issued certificates remain cryptographically valid
Relying-party behavior depends on refresh	Cached trust stores continue trusting until refreshed
Historical records preserved	Audit log retains all history; removal is logged

Table 8: Trust store removal semantics

OpenAuthority does not have the ability to revoke or invalidate certificates issued by a trust anchor. It controls only trust store contents.

9.2 Security Boundary

OpenAuthority cannot technically prevent a constrained root from issuing certificates outside its declared namespace. Security depends on relying parties enforcing Name Constraints during certificate validation.

9.3 Relying Party Responsibilities

- Refresh trust stores regularly (recommended: daily)
- Understand that trust store removal is not certificate revocation
- Implement certificate validation (OCSP, CRL) independently
- Enforce X.509 Name Constraints during path validation

10 Transparency and Auditability

10.1 Audit Log Guarantees

Property	Guarantee
Append-only	Hash chaining; new entries reference previous hash
Tamper-evident	Modifications break the chain
Publicly queryable	Entries and inclusion proofs available via API
Externally anchored	Periodic root hashes anchored to timestamp authorities

Table 9: Audit log guarantees

10.2 Independently Auditable Trust Governance

The combination of append-only logging, Merkle proofs, and external anchoring enables:

- Verification that a trust anchor was included or removed at a specific time
- Reconstruction of historical trust decisions from the log
- Non-repudiable temporal attestation via external anchors
- Deterministic derivation of trust store contents from logged state

11 Merkle Structure

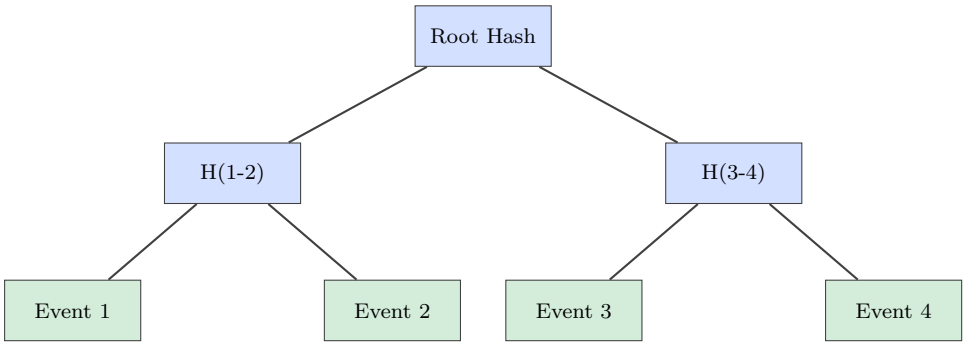

Figure 3: Merkle tree structure

- **Leaf nodes**: Hashes of individual verification events
- **Internal nodes**: Hashes of child nodes
- **Root hash**: Represents entire log state

Inclusion proofs allow third parties to verify event inclusion without downloading the entire log.

Consistency proofs demonstrate the log is append-only—newer tree heads are valid extensions of older ones.

12 External Anchoring

12.1 RFC 3161 Timestamps

Merkle root submitted to Time Stamping Authorities; signed token binds hash to specific time.

12.2 Rekor Integration

Alternative anchoring to Sigstore's Rekor transparency log provides:

- Public verifiability of submissions
- Integration with supply chain security ecosystem
- Redundant temporal attestation

13 Architectural Tradeoffs

Tradeoff	**Consequence**
Continuous verification	Increased operational complexity; verification infrastructure must remain available
DNS-based eligibility	Dependency on DNS infrastructure integrity; trust dependency shifts rather than disappears
Frequent trust updates	Relying parties must refresh trust stores to reflect changes
Mandatory Name Constraints	Reduced flexibility compared to unconstrained roots
Probationary period	7-day delay before trust anchor activation
Grace period on failure	72-hour window where failed CA remains in exports

Table 10: Architectural tradeoffs

These tradeoffs are intentional. They prioritize scoped authority and automatic decay over operational convenience.

14 Operational Constraints

14.1 Trust Anchor Requirements

Constraint	Requirement
Name Constraints	Mandatory
Key Algorithm	RSA (2048+) or ECDSA (P-256, P-384)
DNS Record	Must maintain TXT at `_openauthority.<domain>`

Table 11: Trust anchor requirements

14.2 Security Controls

- Defensive ASN.1 parsing with depth and size limits
- Hardened DNS packet parsing
- Resource exhaustion protections
- Audit log integrity verification on startup

15 Limitations

15.1 Experimental Status

Warning

This software is experimental and has not undergone formal security review. Do not rely on it for production-critical trust decisions without independent evaluation.

15.2 Known Limitations

Limitation	Description
DNS Dependency	Security relies on DNS infrastructure integrity
No DNSSEC Requirement	Currently operates independent of DNSSEC; integration is future work
No Browser Integration	Requires explicit trust store installation
Single Operator	Current deployment is centralized
Detection, Not Prevention	Cannot prevent out-of-scope issuance—only remove from trust store
No Certificate Revocation	Removal does not invalidate already-issued certificates
Verification Latency	DNS changes may not be detected immediately
Name Constraint Enforcement Varies	Effectiveness depends on relying-party software correctly enforcing constraints

Table 12: Known limitations

16 Future Research Directions

- DNSSEC validation to strengthen DNS-based proofs
- Multi-operator verification for reduced single points of failure
- Formal security proofs for the verification model
- Independent security audit
- Client libraries for trust store consumption

17 Target Audience

Audience	Use Case
Security Researchers	Exploring constrained trust models
Homelabbers	Operating private CAs with verifiable boundaries
Enterprises	Internal PKI with scoped authority and automatic decay
Infrastructure Operators	Systems requiring auditable trust relationships

Table 13: Target audience

OpenAuthority is a research platform for exploring whether least-privilege principles can be meaningfully applied to certificate authority trust.

18 Conclusion

OpenAuthority operationalizes a simple principle: certificate authority trust should be scoped, time-bounded, and continuously verified.

The central contribution is treating trust eligibility as a continuously evaluated state rather than a permanent binary. By combining mandatory name constraints, continuous DNS-based verification, and cryptographically protected transparency logs, it demonstrates a model where:

- Trust authority is scoped to verified namespaces
- Authority decays automatically when eligibility proofs disappear
- All trust decisions are observable and auditable

While not intended as production infrastructure, OpenAuthority provides a research platform for investigating whether least-privilege principles can meaningfully reduce the operational risks of certificate authority trust.

References

- RFC 5280: Internet X.509 PKI Certificate and CRL Profile
- RFC 3161: Internet X.509 PKI Time-Stamp Protocol
- RFC 6962: Certificate Transparency
- Sigstore Rekor: https://docs.sigstore.dev/logging/overview/

License

Revision History

Version	Changes
v0.1	Initial draft
v0.2	Added least-privilege framing; stale delegation threat; lifecycle examples; failure semantics
v0.3	Added terminology section; clarified DNS as operational proxy; trust store semantics; transparency positioning
v0.4	Trimmed repetition (~15%); added Tradeoffs section; added state machine diagram
v0.5	Added Name Constraint enforcement caveat; clarified security boundary; added "Trust Freshness" terminology; added relying party model; sharpened CT differentiation; elevated "continuously evaluated state" as central innovation
v0.6	Added explicit DNSSEC rationale; operationalized trust freshness with maximum freshness windows; added DNSSEC to limitations table
v1.0	Added explicit statement that OpenAuthority reduces scope rather than eliminates trust dependencies; clarified DNS tradeoff as trust shift; finalized for publication

Table 14: Document revision history